

Sequence Mining: Terminology

Let Σ be the *alphabet*, a set of symbols. A *sequence* or a *string* is defined as an ordered list of symbols, and is written as $\mathbf{s} = s_1 s_2 \dots s_k$, where $s_i \in \Sigma$ is a symbol at position i , also denoted as $\mathbf{s}[i]$. $|\mathbf{s}| = k$ denotes the *length* of the sequence.

The notation $\mathbf{s}[i : j] = s_i s_{i+1} \dots s_{j-1} s_j$ denotes the *substring* or sequence of consecutive symbols in positions i through j , where $j > i$.

Define the *prefix* of a sequence $\mathbf{s}$ as any substring of the form $\mathbf{s}[1 : i] = s_1 s_2 \dots s_i$, with $0 \leq i \leq n$.

Define the *suffix* of $\mathbf{s}$ as any substring of the form $\mathbf{s}[i : n] = s_i s_{i+1} \dots s_n$, with $1 \leq i \leq n + 1$.

$\mathbf{s}[1 : 0]$ is the empty prefix, and $\mathbf{s}[n + 1 : n]$ is the empty suffix. Let Σ^* be the set of all possible sequences that can be constructed using the symbols in Σ , including the empty sequence $\emptyset$ (which has length zero).

Sequence Mining: Terminology

Let $\mathbf{s} = s_1 s_2 \dots s_n$ and $\mathbf{r} = r_1 r_2 \dots r_m$ be two sequences over Σ . We say that $\mathbf{r}$ is a *subsequence* of $\mathbf{s}$ denoted $\mathbf{r} \subseteq \mathbf{s}$, if there exists a one-to-one mapping $\phi : [1, m] \rightarrow [1, n]$, such that $\mathbf{r}[i] = \mathbf{s}[\phi(i)]$ and for any two positions i, j in $\mathbf{r}$, $i < j \implies \phi(i) < \phi(j)$. In If $\mathbf{r} \subseteq \mathbf{s}$, we also say that $\mathbf{s}$ *contains* $\mathbf{r}$.

The sequence $\mathbf{r}$ is called a *consecutive subsequence* or substring of $\mathbf{s}$ provided $r_1 r_2 \dots r_m = s_j s_{j+1} \dots s_{j+m-1}$, i.e., $\mathbf{r}[1 : m] = \mathbf{s}[j : j + m - 1]$, with $1 \leq j \leq n - m + 1$.

Let $\Sigma = \{A, C, G, T\}$, and let $\mathbf{s} = ACTGAACG$.

Then $\mathbf{r}_1 = CGAAG$ is a subsequence of $\mathbf{s}$, and $\mathbf{r}_2 = CTGA$ is a substring of $\mathbf{s}$. The sequence $\mathbf{r}_3 = ACT$ is a prefix of $\mathbf{s}$, and so is $\mathbf{r}_4 = ACTGA$, whereas $\mathbf{r}_5 = GAACG$ is one of the suffixes of $\mathbf{s}$.

Frequent Sequences

Given a database $\mathbf{D} = \{\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_N\}$ of N sequences, and given some sequence $\mathbf{r}$, the *support* of $\mathbf{r}$ in the database $\mathbf{D}$ is defined as the total number of sequences in $\mathbf{D}$ that contain $\mathbf{r}$

$$\text{sup}(\mathbf{r}) = \left| \{ \mathbf{s}_i \in \mathbf{D} \mid \mathbf{r} \subseteq \mathbf{s}_i \} \right|$$

The *relative support* of $\mathbf{r}$ is the fraction of sequences that contain $\mathbf{r}$

$$\text{rsup}(\mathbf{r}) = \text{sup}(\mathbf{r}) / N$$

Given a user-specified *minsup* threshold, we say that a sequence $\mathbf{r}$ is *frequent* in database $\mathbf{D}$ if $\text{sup}(\mathbf{r}) \geq \text{minsup}$. A frequent sequence is *maximal* if it is not a subsequence of any other frequent sequence, and a frequent sequence is *closed* if it is not a subsequence of any other frequent sequence with the same support.

Mining Frequent Sequences

For sequence mining the order of the symbols matters, and thus we have to consider all possible *permutations* of the symbols as the possible frequent candidates. Contrast this with itemset mining, where we had only to consider *combinations* of the items.

The sequence search space can be organized in a prefix search tree. The root of the tree, at level 0, contains the empty sequence, with each symbol $x \in \Sigma$ as one of its children. As such, a node labeled with the sequence $\mathbf{s} = s_1 s_2 \dots s_k$ at level k has children of the form $\mathbf{s}' = s_1 s_2 \dots s_k s_{k+1}$ at level $k + 1$. In other words, $\mathbf{s}$ is a prefix of each child $\mathbf{s}'$, which is also called an *extension* of $\mathbf{s}$.

Example Sequence Database

Id	Sequence
s_1	CAGAAGT
s_2	TGACAG
s_3	GAAGT

Using $minsup = 3$, the set of frequent subsequences is given as:

$$\mathcal{F}^{(1)} = A(3), G(3), T(3)$$

$$\mathcal{F}^{(2)} = AA(3), AG(3), GA(3), GG(3)$$

$$\mathcal{F}^{(3)} = AAG(3), GAA(3), GAG(3)$$

$$\mathcal{F}^{(4)} = GAAG(3)$$

Level-wise Sequence Mining: GSP Algorithm

The GSP algorithm searches the sequence prefix tree using a level-wise or breadth-first search. Given the set of frequent sequences at level k , we generate all possible sequence extensions or *candidates* at level $k + 1$. We next compute the support of each candidate and prune those that are not frequent. The search stops when no more frequent extensions are possible.

The prefix search tree at level k is denoted $\mathcal{C}^{(k)}$. Initially $\mathcal{C}^{(1)}$ comprises all the symbols in Σ . Given the current set of candidate k -sequences $\mathcal{C}^{(k)}$, the method first computes their support.

For each database sequence $s_i \in \mathbf{D}$, we check whether a candidate sequence $r \in \mathcal{C}^{(k)}$ is a subsequence of s_i . If so, we increment the support of r . Once the frequent sequences at level k have been found, we generate the candidates for level $k + 1$.

For the extension, each leaf r_a is extended with the last symbol of any other leaf r_b that shares the same prefix (i.e., has the same parent), to obtain the new candidate $(k + 1)$ -sequence $r_{ab} = r_a + r_b[k]$. If the new candidate r_{ab} contains any infrequent k -sequence, we prune it.

Algorithm GSP

GSP (D, Σ, minsup):

```
1  $\mathcal{F} \leftarrow \emptyset$ 
2  $\mathcal{C}^{(1)} \leftarrow \{\emptyset\}$  // Initial prefix tree with single symbols
3 foreach  $s \in \Sigma$  do Add  $s$  as child of  $\emptyset$  in  $\mathcal{C}^{(1)}$  with  $\text{sup}(s) \leftarrow 0$ 
4  $k \leftarrow 1$  //  $k$  denotes the level
5 while  $\mathcal{C}^{(k)} \neq \emptyset$  do
6   | ComputeSupport ( $\mathcal{C}^{(k)}, D$ )
7   | foreach leaf  $s \in \mathcal{C}^{(k)}$  do
8   |   | if  $\text{sup}(r) \geq \text{minsup}$  then  $\mathcal{F} \leftarrow \mathcal{F} \cup \{(r, \text{sup}(r))\}$ 
9   |   | else remove  $s$  from  $\mathcal{C}^{(k)}$ 
10  |  $\mathcal{C}^{(k+1)} \leftarrow \text{ExtendPrefixTree}(\mathcal{C}^{(k)})$ 
11  |  $k \leftarrow k + 1$ 
12 return  $\mathcal{F}^{(k)}$ 
```